

Workshop on Computational Fluid Dynamics (CFD)
with OpenFOAM
IMP-PAN Gdańsk

Tutorial #3: Compressible air Ejector
Supersonic Transient and Steady Flow

Robert Castilla
CATMech - Universitat Politècnica de Catalunya
Department of Fluid Mechanics

2025-26
Version 1.0

Contents

1	Introduction and Theoretical Background	3
1.1	Case Overview	4
2	Case Setup	4
3	Part A: Density based explicit solver <code>rhoCentralFoam</code>	4
3.1	Running the Simulation	4
3.1.1	Mesh generation	5
3.1.2	Initialization and solver execution	5
3.2	Post-Processing in Terminal	5
3.3	Visualization in paraview	5
4	Part B: Density based implicit solver <code>HiSA</code>	5
4.1	Post-Processing in Terminal	5
4.2	Visualization with paraview	5
5	Student Exercises and Extensions	5
5.1	Basic exercises for Part A: VOF	5

5.2 Basic exercises for Part B: Euler-Euler	5
---	---

Learning Objectives

By the end of this tutorial the student will be able to

1. **Navigate** OpenFOAM case directory structure
2. **Understand** basic OpenFOAM file formats
3. **Run** a simple transient simulation
4. **Analyze** basic results in terminal and **paraview**

1 Introduction and Theoretical Background

In this session a supersonic air ejector is simulated. In this case high velocity air is used to create a very low pressure region with a shock wave. after this first shock wave secondary flow air is suctioned and mixed with the main jet. This mixed flow is accelerated in the mixing chamber until a second shock wave is created in the diffuser.

This simulation is challenging because of the high Ma number compressible flow. For the previous sessions, where fluid were either liquids or low velocity gas, the simulations were performed by using a **pressure-based solver** (SIMPLE, PISO or PIMPLE algorithm). Momentum balance is computed by defining an equation for pressure (Poisson equation) that is corrected with continuity and then again computed with momentum equation until convergence. This method is not valid for high Ma number flows, since it is not capable to adequately resolve shock waves. Instead, a **density-based solver** is used. These solvers consider energy equation, computing pressure from density and perfect gas equation (or any other gas state equation).

Two different approaches for density-based solvers will be considered:

- **rhoCentralFoam** is a transient **explicit** density-based solver.
- **HiSA** is a steady **implicit** density-based solver.¹

The implicit approach is more stable and robust, but, on the other hand, it requires a more complex solver configuration and is more computationally demanding, both in time and memory.

¹It can also be configured to simulate a transient flow, but in this session only the steady state case is treated.

1.1 Case Overview

The scheme of the ejector is depicted in Figure 1. Note that the primary nozzle has now the typical convergence-divergent shape in order to speed up the primary flow and get a shock wave. Also, the outlet has been enlarged in order to avoid as much as possible pressure wave reflections.

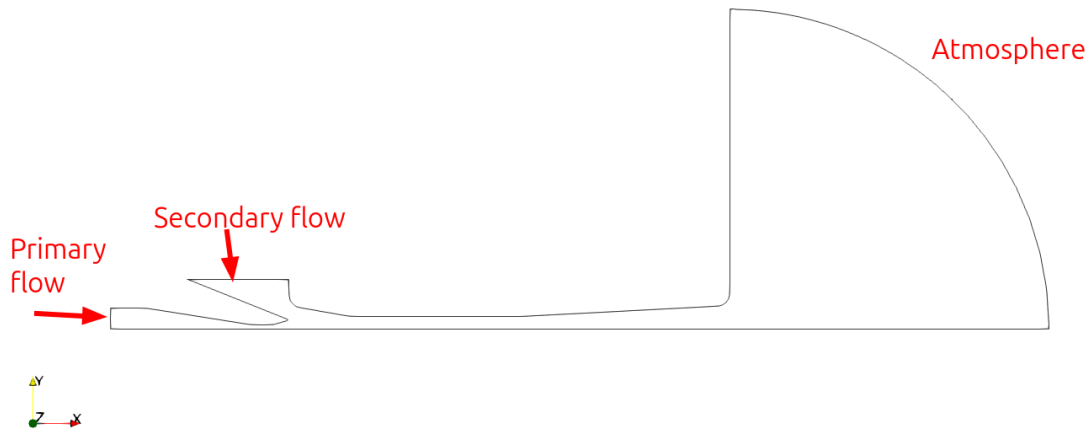


Figure 1: Scheme of supersonic ejector

2 Case Setup

The python environment used in Session 2 has to be also activated for the present tutorial.

3 Part A: Density based explicit solver `rhoCentralFoam`

The clean case can be downloaded from the GitHub repository

3.1 Running the Simulation

Again, the simulation is split in two parts: the generation of the mesh and running the solver.

3.1.1 Mesh generation

To generate the `blockMeshDict` file, run

Check that the python virtual environment is activated and run

```
python supersonic_blockMeshDictGen.py
```

The `blockMeshDict` is created in the same folder. Move to the `system` folder of the case to be simulated (now the `rCF` case).

The `genMesh.scr` script runs the `blockMesh`, `extrudeMesh` and `checkMesh` commands.

3.1.2 Initialization and solver execution

To run the solver, type

```
./runCase.scr
```

This simulation is very slow. It takes about 4 to 5 hours to run 5 ms. But this is enough to get a steady state flow. Hoew

3.2 Post-Processing in Terminal

3.3 Visualization in paraview

4 Part B: Density based implicit solver **HiSA**

4.1 Post-Processing in Terminal

4.2 Visualization with paraview

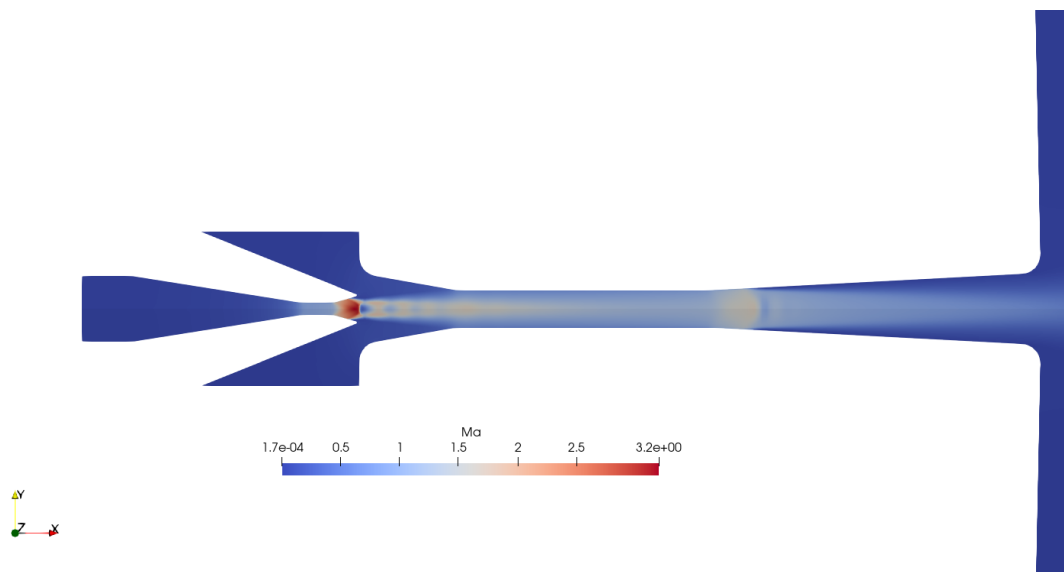


Figure 2: Mach number distribution

5 Student Exercises and Extensions

5.1 Basic exercises for Part A: VOF

5.2 Basic exercises for Part B: Euler-Euler

Disclaimer

This tutorial has been developed by **Robert Castilla**, of the Departament of Fluid Mechanics in the Universitat Politècnica de Catalunya, for the doctoral workshop at **IMPPAN Gdańsk** (July 2026). The author utilized **DeepSeek** for the initial definition of the document format. Further technical refinement and the development of advanced post-processing exercises were supported by **NotebookLM**. All the technical information

for the **OpenFOAM v2512** environmnet and all physical interpretations were performed and approved by the author.